

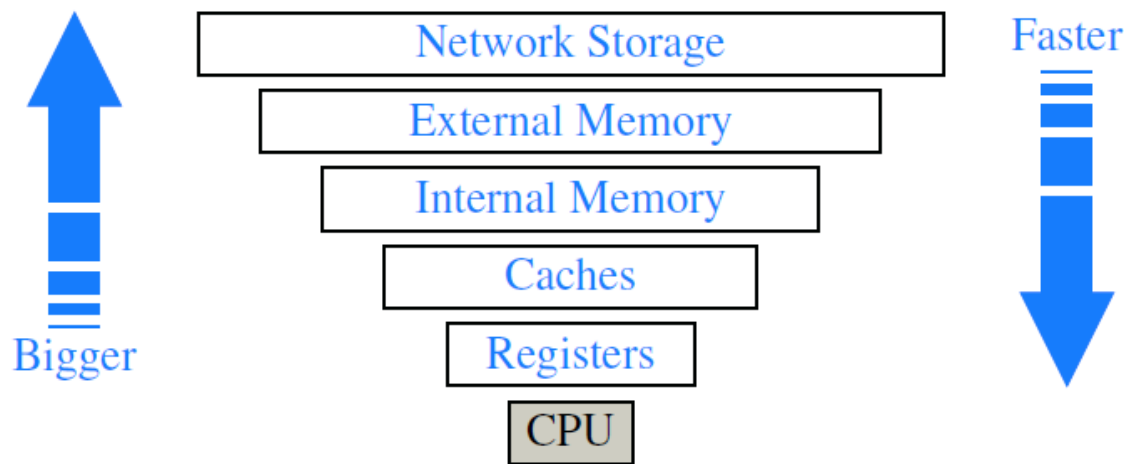
B-ALBERI

Giovedì 26 marzo 2026



MEMORY SYSTEM

- Per memorizzare grandi dataset, i computer usano una gerarchia di tipi differenti di memoria, che variano in termini della loro taglia e dalla distanza dalla CPU.



- Durante l'esecuzione di un programma, i dati sono copiati da un livello di memoria al successivo. Questi trasferimenti diventano il vero collo di bottiglia per il tempo di esecuzione.



DATI IN MEMORIA SECONDARIA

- Le moderne piattaforme di calcolo contengono una gerarchia di livelli di memoria, ciascuno con il proprio costo e le proprie caratteristiche prestazionali.
- La gerarchia spazia da memorie veloci (ordine dei nanosecondi per un accesso), costose e con una piccola capacità (come i registri e le cache, ordine di pochi Megabyte), a memorie lente (millisecondi) poco costose e grandi (come i dischi, ordine di decine di Terabyte).
- Le applicazioni relative a motori di ricerca per il Web, basi di dati, sistemi informativi geografici etc., devono gestire quantità di dati così grandi che la memoria centrale è insufficiente a contenerli tutti.
- Recuperare un dato da disco è molto più lento che accedere alla memoria centrale. E' quindi importante minimizzare il numero di letture e scritture su memoria esterna (I/O). Per diminuire e ammortizzare il tempo speso ad ogni accesso, i dati vengono letti da disco – e portati in memoria centrale – in blocchi di dimensione B più o meno grande.
- E' quindi importante che i dati siano importati tramite un singolo accesso al disco, fintantoché si trovano in memoria centrale, siano «sfruttati» il più possibile da un dato algoritmo.
- In altre parole, se un algoritmo esibisce proprietà di località nell'accesso ai dati, il numero di I/O viene minimizzato, ottenendo prestazioni migliori.



GERARCHIE DI MEMORIA E PERFORMANCE DI UN ALGORITMO

- L'influenza delle gerarchie di memoria sulle performance di un programma dipende fortemente dalla taglia del problema che intendiamo risolvere e dalle caratteristiche fisiche del computer system.
- Spesso, il collo di bottiglia si riferisce a due livelli di memoria consecutive, quello che è in grado di memorizzare i dati e quello immediatamente precedente.
- Per un problema che può interamente essere gestito nella memoria principale, i due livelli di memoria più importanti sono: la memoria cache e la memoria interna. I tempi di accesso per la memoria interna possono essere 10 o 100 volte più lenti di quelli per la memoria cache.
- Per un problema che non “entra” interamente nella memoria interna, i due livelli di memoria più importanti sono: la memoria esterna e la memoria interna. Qui la differenza è più significativa. I tempi di accesso ad un disco esterno sono tipicamente da 100.000 a 1.000.000 volte più lenti rispetto a quelli per la memoria interna. L'obiettivo diventa fare il minor numero possibile di accessi alla memoria esterna.
- Molti algoritmi però non sono progettati avendo in mente le gerarchie di memoria! Spesso le analisi degli algoritmi assumono che i tempi di accesso siano uguali per tutti i tipi di memoria.



IL MODELLO UTILIZZATO

- La memoria è suddivisa in pagine
- Pagina (o Page): blocco contiguo di dati (per esempio, un file o un blocco da 4096 byte);
- Sondaggio (o Probe): Primo accesso alla pagina (per esempio, dal disco alla memoria);
- Proprietà: Il tempo richiesto per un probe è molto più grande del tempo di accesso ai dati in una pagina;
- Modello di costo: Numero di probe;
- Obiettivo: Accedere ai dati usando il numero minimo di probe (o in modo equivalente, il minor numero di trasferimenti di blocchi – I/O complexity). Ciò implica per esempio effettuare ricerca e inserimento su tabelle di simboli con migliaia di miliardi di oggetti, usando quattro o cinque riferimenti a piccoli blocchi di dati.



ALCUNE RAPPRESENTAZIONI INEFFICIENTI IN MEMORIA ESTERNA

- Se rappresentiamo n elementi in una lista doppiamente linkata, la ricerca di una chiave nella lista richiede n trasferimenti nel caso peggiore poiché è abbastanza verosimile che ogni elemento della lista faccia riferimento a differenti blocchi di memoria.
- Possiamo ridurre il numero di trasferimenti di blocchi memorizzando la sequenza in un array. In questo caso una ricerca sequenziale può essere realizzata usando solo $O(n/B)$ trasferimenti di blocchi, a causa della località spaziale della struttura, dove B denota la dimensione del blocco.
- Se la sequenza è memorizzata in modo ordinato in un array, la ricerca binaria effettua $O(\log_2 n)$ trasferimenti, che è un risultato interessante. Ma non si trae alcun beneficio da ogni trasferimento di blocco poiché ogni ricerca binaria può fare riferimento ad un blocco diverso.
- Queste implementazioni sono I/O inefficienti. Si potrebbero considerare alberi bilanciati memorizzati in memoria interna, con un tempo di ricerca logaritmico. Verosimilmente ogni nodo risiede in un blocco diverso.



DIZIONARI IN MEMORIA SECONDARIA

- Le implementazioni dei dizionari finora proposte non esibiscono buona località: intuitivamente non offrono alcuna garanzia che i nodi su un certo cammino di ricerca risiedano nello stesso blocco, e quindi si potrebbero avere $\Theta(\log n)$ operazioni di I/O nel caso peggiore.
- L'idea è quella di aumentare l'informazione a carico di ciascun nodo ed il grado dei nodi rendendoli proporzionali a B.
- In questo modo potremmo usare tutta l'informazione contenuta in ogni blocco che porteremo in memoria principale, e poiché diminuiremo considerevolmente l'altezza dell'albero, avremo un numero minore di accessi a memoria secondaria.
- I B-alberi sono stati introdotti nel 1970 da Bayer e McCreight e rappresentano una naturale estensione degli alberi 2-3 (nella interpretazione 1).



B-ALBERO

- Invece che memorizzare i dati nell'albero, si costruisce un albero con copie delle chiavi, ogni copia è associata ad un link.
- Generalizzazione degli alberi 2-3 per ospitare fino a $M - 1$ coppie chiave-link per nodo (si suppone che M sia un numero pari positivo);
- M viene scelto come il più grande che consenta di riempire una pagina con M link;
- La radice ha almeno 2 coppie chiave-link;
- Ogni altro nodo ha almeno $M/2$ coppie chiave-link;
- Separazione dell'indice dai dati:
 - Nodi interni: copie delle chiavi guidano la ricerca;
 - Nodi esterni: client keys con riferimenti ai dati.



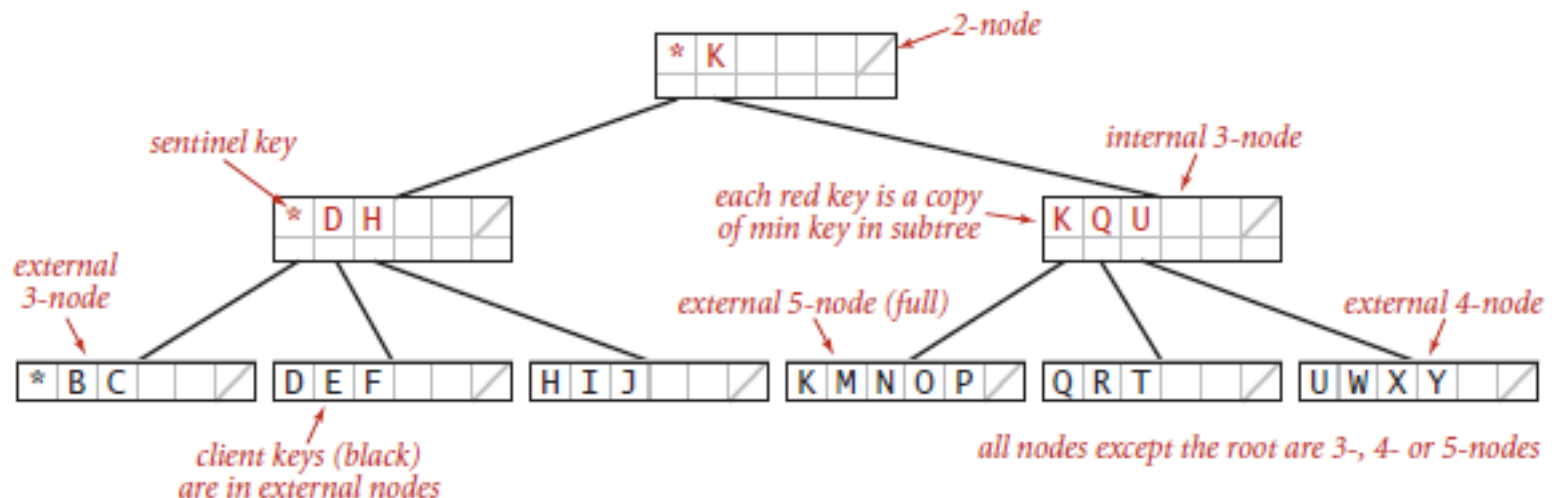
B-ALBERO: DEFINIZIONE

- Un B-albero di ordine M è un k -nodo esterno (con k chiavi), oppure un albero con k -nodi interni (ognuno con k chiavi e k link a B-alberi che rappresentano ciascuno dei k intervalli delimitati dalle chiavi) e k -nodi esterni con le proprietà:
 - tutti i cammini dalla radice ad un nodo esterno hanno la stessa lunghezza (bilanciamento perfetto);
 - $2 \leq k \leq M-1$ alla radice;
 - $M/2 \leq k \leq M-1$ agli altri nodi.
- Ogni chiave in un nodo interno è associata con un altro nodo che è radice di un albero le cui chiavi sono tutte maggiori o uguali della chiave in questione e minori della chiave successiva, se presente.



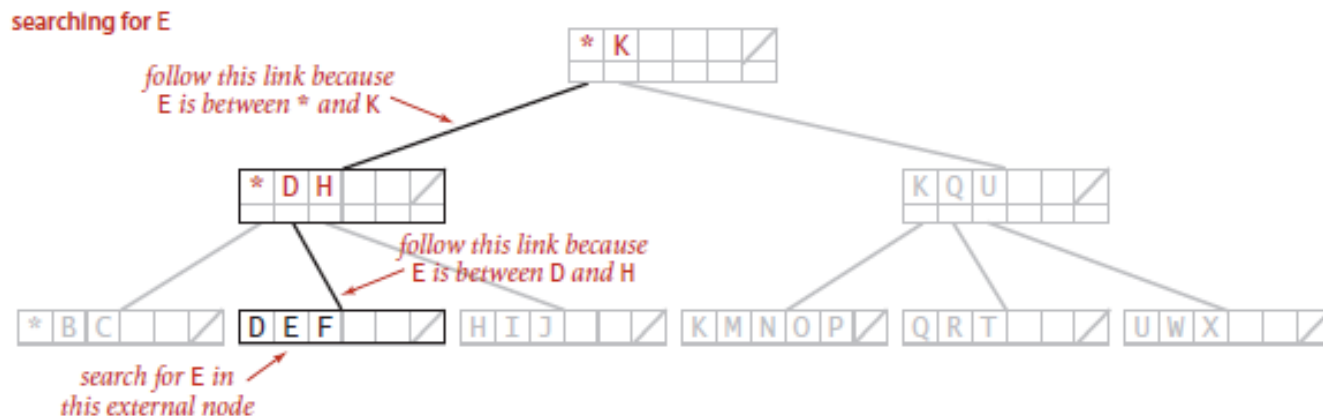
B-ALBERO DI ORDINE 6

- E' conveniente usare una chiave speciale, chiamata sentinella, che è minore di tutte le altre chiavi
- Un nodo è pieno (full) se contiene esattamente $M-1$ chiavi.



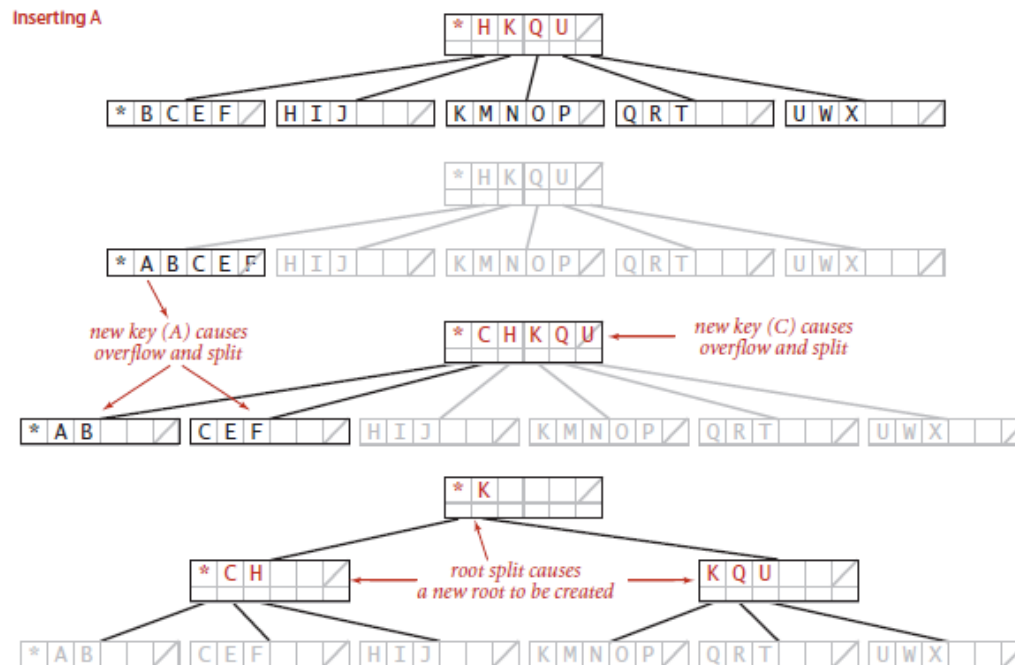
RICERCA IN UN B-ALBERO

- Cerca alla radice;
- Trova un intervallo per la chiave e prendi il link relativo;
- Termina la ricerca in un nodo esterno.
- Nota: Non ci fermiamo ai nodi interni perché così l'implementazione è più semplice, e non c'è svantaggio apprezzabile per tabelle grandi.
- Nota: Uso della sentinella * per semplificare la ricerca



INSERIMENTO IN UN B-ALBERO

- Ricerca la chiave;
- Se non presente, inseriscila al fondo;
- Risalendo verso la radice, dividi i nodi con M chiavi.



B-ALBERI: BILANCIAMENTO

- PROPOSIZIONE: Una ricerca o un inserimento in un B-albero di ordine M con N chiavi richiedono un numero di probe compreso tra $\log_{M-1} N$ e $\log_{\frac{M}{2}} N$.
- Dimostrazione: Segue dal fatto che tutti i nodi interni (eccetto la radice) hanno un numero di figli compreso tra $M/2$ e $M-1$. Il caso peggiore si ha quando tutti i nodi interni hanno $M/2$ figli, nel caso migliore ogni nodo ha $M-1$ figli.
- In pratica: Il numero di probe è inferiore a 4 ($M = 1024; N = 62\text{miliardi}; \log_{\frac{M}{2}} N \leq 4$).
- Ottimizzazione: Si può eliminare un probe mantenendo la radice nella memoria interna.



B-ALBERO: TEMPO

- Nel caso peggiore, poiché ad ogni chiamata si scende di livello, il numero di chiamate ricorsive è pari all'altezza dell'albero, ovvero $O(\log_{\frac{M}{2}} N)$.
- Se in ogni nodo entra in un blocco, il numero di I/O diventa logaritmico. Il vantaggio maggiore si ha quindi se si sceglie M proporzionale a B . Su ogni nodo si spende tempo $O(M)$. Poiché le chiavi sono ordinate si può effettuare una ricerca binaria, ovvero $O(\log M)$
- In totale $O(\log M \cdot \log_{\frac{M}{2}} N)$ ovvero $O(\log N)$



B-ALBERO: SPAZIO

- Si prova che nel caso peggiore, un B-albero usa il doppio dello spazio necessario per le chiavi, più ulteriore spazio per i link. E' stato dimostrato (Yao, 1979) che il numero medio di chiavi in un nodo è circa $M \ln 2$, e lo spazio inutilizzato è mediamente il 44%.



IMPLEMENTAZIONE DI B-ALBERI: RAPPRESENTAZIONE DEI NODI

- Per semplicità usiamo un solo tipo di nodo (per le foglie e per i nodi interni)
- Ogni nodo è formato da un array e ciascuna delle componenti di questo array contiene un elemento, una chiave e un link. Ciascun nodo contiene un contatore che tiene conto degli elementi attivi nel nodo. M rappresenta l'ordine del B-tree
- Si definisce una struct o una classe per i nodi

```
template <typename Item, typename Key>
struct entry
{ Key key; Item item; struct node *next; };
struct node
{ int m; entry<Item, Key> b[M];
  node() { m = 0;}
};
typedef node *link;
```



RICERCA IN UN B-TREE

La ricerca distingue tra nodi interni e foglie

private:

Item searchR(link h, Key v, int ht)

{ int j;

if (ht == 0)

for (j = 0; j < h->m; j++)

{ if (v == h->b[j].key)

return h->b[j].item; }

else

for (j = 0; j < h->m; j++)

if ((j+1 == h->m) || (v < h->b[j+1].key))

return searchR(h->b[j].next, v, ht-1);

return nullItem;

}

public:

Item search(Key v)

{ return searchR(head, v, HT); }

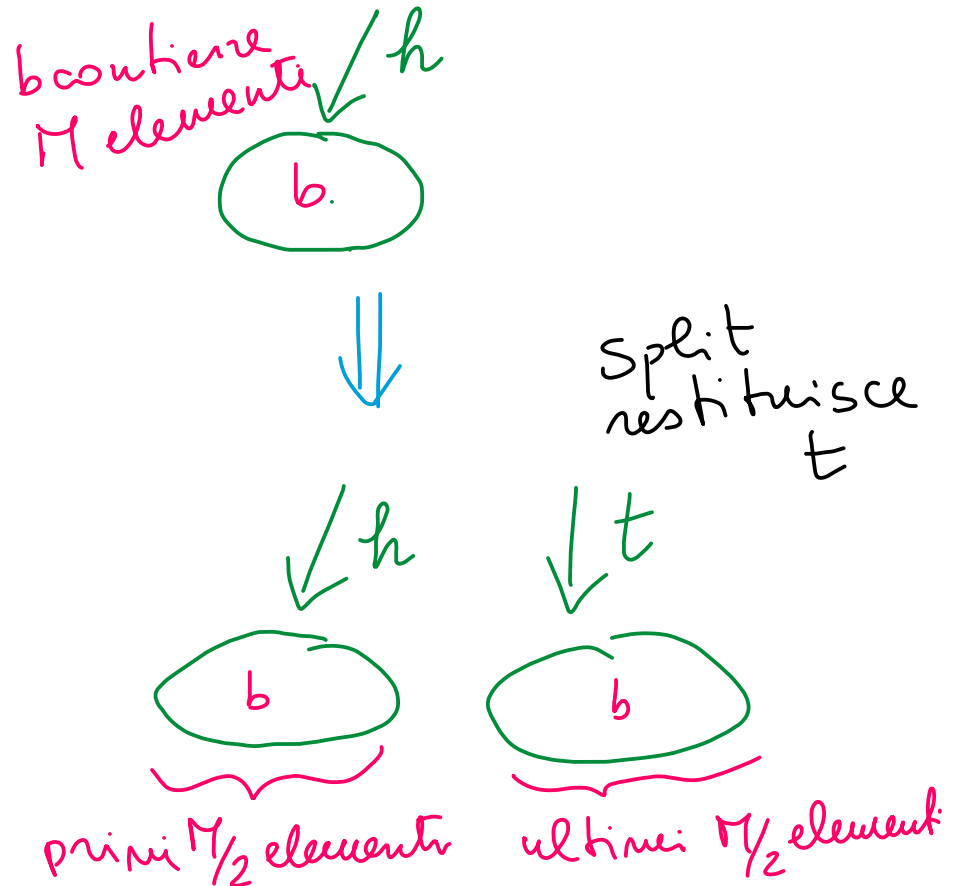
foglia →

*nodo
interno* →



OPERAZIONE DI SPLIT

```
link split(link h)
{ link t = new node();
  for (int j = 0; j < M/2; j++)
    t->b[j] = h->b[M/2+j];
  h->m = M/2;
  t->m = M/2;
  return t; }
```



INSERIMENTO IN UN B-TREE

Item è un tipo di dato che contiene un'informazione e la chiave. Se ci interessa solo la chiave possiamo usare Key

private:

link insertR(link h, Item x, int ht)

{ int i, j; Key v = x.key(); entry<Item, Key> t;

t.key = v; t.item = x;

if (ht == 0)

for (j = 0; j < h->m; j++)

{ if (v < h->b[j].key) break; }

else

for (j = 0; j < h->m; j++)

if ((j+1 == h->m) || (v < h->b[j+1].key))

{ link u;

u = insertR(h->b[j+1].next, x, ht-1);

if (u == NULL) return NULL;

t.key = u->b[0].key;

t.next = u;

break;

for (i = h->m; i > j; i--) h->b[i] = h->b[i-1];

h->b[j] = t;

if (++h->m < M) return NULL;

else return split(h);

}

public:

void insert(Item item)

{ link u = insertR(head, item, HT);

if (u == NULL) return;

link t = new node(); t->m = 2;

t->b[0].key = head->b[0].key;

t->b[1].key = u->b[0].key;

t->b[0].next = head; t->b[1].next = u;

head = t; HT++;

}

eventuale
split
della
radice

(1) All'uscita dal ciclo
j è la posizione dove
effettuare l'inserimento
del nuovo elemento t

(2) j è la posizione dell'albero
in cui proseguire

(3) Se u è NULL non ci sono stati
split nel nodo h e la funzione termina, altrimenti
Incremento j e aggravo t che verrà inserito nel nodo h

